Lecture 3: Architectures: dense, attention variants, MoE

Building Large Foundation Models from Scratch · WS 2026/27

Asst. Prof. Dr. Martin Pawelczyk

Faculty of Computer Science, University of Vienna

Thu 8 Oct 2026

Where we are

Admin, 2 minutes

- Does everyone have access to Moodle?
- Did you start with the tokenizer?
- Do you have access to Google Colab?
- This week you should start with the model!

Recap, 15 minutes

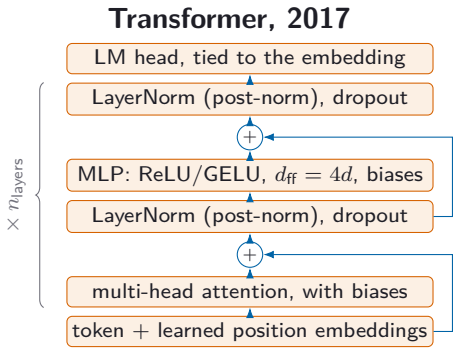
Today

1. The modern Transformer block
2. Hyperparameters and initialisation
3. Training stability
4. Attention variants
5. Mixture of experts
6. Reading a model card
7. Wrap-up

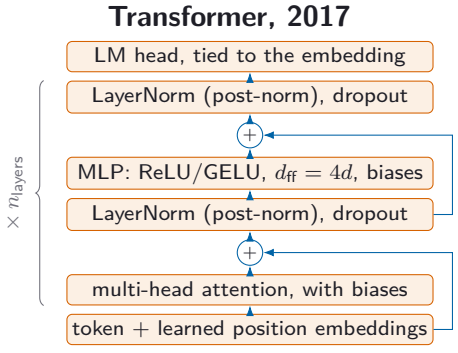
The modern Transformer block

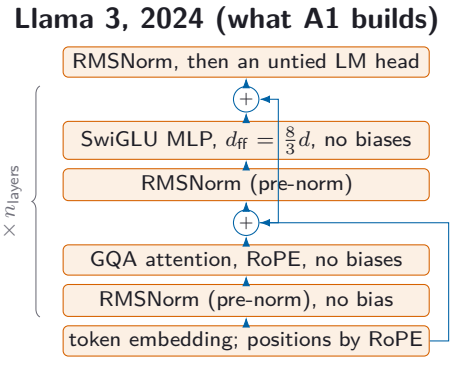
1. The modern Transformer block
2. Hyperparameters and initialisation
3. Training stability
4. Attention variants
5. Mixture of experts
6. Reading a model card
7. Wrap-up

Two popular transformer blocks

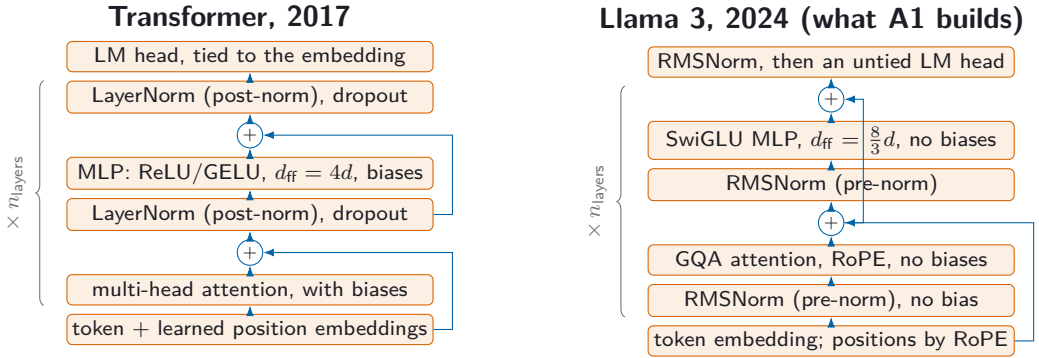


Two popular transformer blocks





Two popular transformer blocks



Orange: changed. Blue: the residual stream and its two adds, the one thing that did not change. No dropout on the right. (GPT-2, 2019: the left stack with pre-norm and GELU.)

The 2024 network end to end

Hyperparameters: vocabulary size V, context L, width $d = d_{\text{model}}$, n_{layers} blocks, n_{heads} query and n_{kv} key/value heads, $d_h = d/n_{\text{heads}}$, d_{ff}, RoPE base Θ. $x \in \mathbb{R}^{B \times L \times d}$ throughout.

parameters	shape
E	$V \times d$
W_Q, $W_O^\top$	$d \times n_{\text{heads}} d_h$
W_K, W_V	$d \times n_{kv} d_h$
W_1, W_3, $W_2^\top$	$d \times d_{\text{ff}}$
g_1, g_2, g_f	d
W_{head}	$d \times V$

§n today's sections

Ln other lectures.

The 2024 network end to end

Hyperparameters: vocabulary size V, context L, width $d = d_{\text{model}}$, n_{layers} blocks, n_{heads} query and n_{kv} key/value heads, $d_h = d/n_{\text{heads}}$, d_{ff}, RoPE base Θ. $x \in \mathbb{R}^{B \times L \times d}$ throughout.

1. $x = E[\text{ids}]$, the embedding lookup §2 init

parameters	shape
E	$V \times d$
W_Q, $W_O^\top$	$d \times n_{\text{heads}} d_h$
W_K, W_V	$d \times n_{kv} d_h$
W_1, W_3, $W_2^\top$	$d \times d_{\text{ff}}$
g_1, g_2, g_f	d
W_{head}	$d \times V$

§n today's sections

Ln other lectures.

The 2024 network end to end

Hyperparameters: vocabulary size V, context L, width $d = d_{\text{model}}$, n_{layers} blocks, n_{heads} query and n_{kv} key/value heads, $d_h = d/n_{\text{heads}}$, d_{ff}, RoPE base Θ. $x \in \mathbb{R}^{B \times L \times d}$ throughout.

1. $x = E[\text{ids}]$, the embedding lookup §2 init
   for each of the n_{layers} blocks:

parameters	shape
E	$V \times d$
W_Q, $W_O^\top$	$d \times n_{\text{heads}} d_h$
W_K, W_V	$d \times n_{kv} d_h$
W_1, W_3, $W_2^\top$	$d \times d_{\text{ff}}$
g_1, g_2, g_f	d
W_{head}	$d \times V$

§n today's sections
Ln other lectures.

The 2024 network end to end

Hyperparameters: vocabulary size V, context L, width $d = d_{\text{model}}$, n_{layers} blocks, n_{heads} query and n_{kv} key/value heads, $d_h = d/n_{\text{heads}}$, d_{ff}, RoPE base Θ. $x \in \mathbb{R}^{B \times L \times d}$ throughout.

1. $x = E[\text{ids}]$, the embedding lookup §2 init
   for each of the n_{layers} blocks:
2. $\tilde{x} = \text{RMSNorm}(x;\, g_1)$ §1 norm

parameters	shape
E	$V \times d$
W_Q, $W_O^\top$	$d \times n_{\text{heads}} d_h$
W_K, W_V	$d \times n_{kv} d_h$
W_1, W_3, $W_2^\top$	$d \times d_{\text{ff}}$
g_1, g_2, g_f	d
W_{head}	$d \times V$

§n today's sections

Ln other lectures.

The 2024 network end to end

Hyperparameters: vocabulary size V, context L, width $d = d_{\text{model}}$, n_{layers} blocks, n_{heads} query and n_{kv} key/value heads, $d_h = d/n_{\text{heads}}$, d_{ff}, RoPE base Θ. $x \in \mathbb{R}^{B\times L\times d}$ throughout.

1. $x = E[\text{ids}]$, the embedding lookup §2 init

   for each of the n_{layers} blocks:
2. $\tilde{x} = \text{RMSNorm}(x;\, g_1)$ §1 norm
3. $q = \tilde{x}W_Q$, $k = \tilde{x}W_K$, $v = \tilde{x}W_V$; k, v have only n_{kv} heads §1 GQA
4. split into n_{heads} heads; RoPE(Θ) rotates q, k by position §1 RoPE

parameters	shape
E	$V \times d$
W_Q, $W_O^\top$	$d \times n_{\text{heads}} d_h$
W_K, W_V	$d \times n_{kv} d_h$
W_1, W_3, $W_2^\top$	$d \times d_{\text{ff}}$
g_1, g_2, g_f	d
W_{head}	$d \times V$

§n today's sections

Ln other lectures.

The 2024 network end to end

Hyperparameters: vocabulary size V, context L, width $d = d_{\text{model}}$, n_{layers} blocks, n_{heads} query and n_{kv} key/value heads, $d_h = d/n_{\text{heads}}$, d_{ff}, RoPE base Θ. $x \in \mathbb{R}^{B\times L\times d}$ throughout.

1. $x = E[\text{ids}]$, the embedding lookup — §2 init

   for each of the n_{layers} blocks:
2. $\tilde{x} = \text{RMSNorm}(x;\, g_1)$ — §1 norm
3. $q = \tilde{x}W_Q$, $k = \tilde{x}W_K$, $v = \tilde{x}W_V$; k, v have only n_{kv} heads — §1 GQA
4. split into n_{heads} heads; RoPE(Θ) rotates q, k by position — §1 RoPE
5. $P = \text{softmax}(qk^\top/\sqrt{d_h} + \text{mask})$, fp32; $o = Pv$; merge heads — §4 L4

parameters	shape
E	$V \times d$
W_Q, $W_O^\top$	$d \times n_{\text{heads}} d_h$
W_K, W_V	$d \times n_{kv} d_h$
W_1, W_3, $W_2^\top$	$d \times d_{\text{ff}}$
g_1, g_2, g_f	d
W_{head}	$d \times V$

§n today's sections

Ln other lectures.

The 2024 network end to end

Hyperparameters: vocabulary size V, context L, width $d = d_{\text{model}}$, n_{layers} blocks, n_{heads} query and n_{kv} key/value heads, $d_h = d/n_{\text{heads}}$, d_{ff}, RoPE base Θ. $x \in \mathbb{R}^{B\times L\times d}$ throughout.

1. $x = E[\text{ids}]$, the embedding lookup — §2 init

   for each of the n_{layers} blocks:
2. $\tilde{x} = \text{RMSNorm}(x;\, g_1)$ — §1 norm
3. $q = \tilde{x}W_Q$, $k = \tilde{x}W_K$, $v = \tilde{x}W_V$; k, v have only n_{kv} heads — §1 GQA
4. split into n_{heads} heads; RoPE(Θ) rotates q, k by position — §1 RoPE
5. $P = \text{softmax}(qk^\top/\sqrt{d_h} + \text{mask})$, fp32; $o = Pv$; merge heads — §4 L4
6. $x \leftarrow x + oW_O$

parameters	shape
E	$V \times d$
W_Q, $W_O^\top$	$d \times n_{\text{heads}} d_h$
W_K, W_V	$d \times n_{kv} d_h$
W_1, W_3, $W_2^\top$	$d \times d_{\text{ff}}$
g_1, g_2, g_f	d
W_{head}	$d \times V$

§n today's sections

Ln other lectures.

The 2024 network end to end

Hyperparameters: vocabulary size V, context L, width $d = d_{\text{model}}$, n_{layers} blocks, n_{heads} query and n_{kv} key/value heads, $d_h = d/n_{\text{heads}}$, d_{ff}, RoPE base Θ. $x \in \mathbb{R}^{B\times L\times d}$ throughout.

1. $x = E[\text{ids}]$, the embedding lookup — §2 init

   for each of the n_{layers} blocks:
2. $\tilde{x} = \text{RMSNorm}(x;\, g_1)$ — §1 norm
3. $q = \tilde{x}W_Q$, $k = \tilde{x}W_K$, $v = \tilde{x}W_V$; k, v have only n_{kv} heads — §1 GQA
4. split into n_{heads} heads; RoPE(Θ) rotates q, k by position — §1 RoPE
5. $P = \text{softmax}(qk^\top/\sqrt{d_h} + \text{mask})$, fp32; $o = Pv$; merge heads — §4 L4
6. $x \leftarrow x + o\,W_O$
7. $\tilde{x} = \text{RMSNorm}(x;\, g_2)$

parameters	shape
E	$V \times d$
W_Q, $W_O^\top$	$d \times n_{\text{heads}} d_h$
W_K, W_V	$d \times n_{kv} d_h$
W_1, W_3, $W_2^\top$	$d \times d_{\text{ff}}$
g_1, g_2, g_f	d
W_{head}	$d \times V$

§n today's sections

Ln other lectures.

The 2024 network end to end

Hyperparameters: vocabulary size V, context L, width $d = d_{\text{model}}$, n_{layers} blocks, n_{heads} query and n_{kv} key/value heads, $d_h = d/n_{\text{heads}}$, d_{ff}, RoPE base Θ. $x \in \mathbb{R}^{B\times L\times d}$ throughout.

1. $x = E[\text{ids}]$, the embedding lookup §2 init

   for each of the n_{layers} blocks:
2. $\tilde{x} = \text{RMSNorm}(x;\, g_1)$ §1 norm
3. $q = \tilde{x}W_Q$, $k = \tilde{x}W_K$, $v = \tilde{x}W_V$; k, v have only n_{kv} heads §1 GQA
4. split into n_{heads} heads; RoPE(Θ) rotates q, k by position §1 RoPE
5. $P = \text{softmax}(qk^\top/\sqrt{d_h} + \text{mask})$, fp32; $o = Pv$; merge heads §4 L4
6. $x \leftarrow x + o\,W_O$
7. $\tilde{x} = \text{RMSNorm}(x;\, g_2)$
8. $x \leftarrow x + \big(\text{SiLU}(\tilde{x}W_1) \odot \tilde{x}W_3\big)\, W_2$ §1 SwiGLU §5 MoE

parameters	shape
E	$V \times d$
W_Q, $W_O^\top$	$d \times n_{\text{heads}} d_h$
W_K, W_V	$d \times n_{kv} d_h$
W_1, W_3, $W_2^\top$	$d \times d_{\text{ff}}$
g_1, g_2, g_f	d
W_{head}	$d \times V$

§n today's sections

Ln other lectures.

The 2024 network end to end

Hyperparameters: vocabulary size V, context L, width $d = d_{\text{model}}$, n_{layers} blocks, n_{heads} query and n_{kv} key/value heads, $d_h = d/n_{\text{heads}}$, d_{ff}, RoPE base Θ. $x \in \mathbb{R}^{B \times L \times d}$ throughout.

1. $x = E[\text{ids}]$, the embedding lookup — §2 init

   for each of the n_{layers} blocks:
2. $\tilde{x} = \text{RMSNorm}(x;\, g_1)$ — §1 norm
3. $q = \tilde{x}W_Q$, $k = \tilde{x}W_K$, $v = \tilde{x}W_V$; k, v have only n_{kv} heads — §1 GQA
4. split into n_{heads} heads; RoPE(Θ) rotates q, k by position — §1 RoPE
5. $P = \text{softmax}(qk^\top/\sqrt{d_h} + \text{mask})$, fp32; $o = Pv$; merge heads — §4 L4
6. $x \leftarrow x + o\,W_O$
7. $\tilde{x} = \text{RMSNorm}(x;\, g_2)$
8. $x \leftarrow x + \big(\text{SiLU}(\tilde{x}W_1) \odot \tilde{x}W_3\big)\, W_2$ — §1 SwiGLU §5 MoE

   after the last block:
9. logits $= \text{RMSNorm}(x;\, g_f)\, W_{\text{head}}$, untied — §2 tying
10. loss: cross-entropy in fp32; AdamW, warmup + cosine, clipping — §2–3

parameters	shape
E	$V \times d$
W_Q, $W_O^\top$	$d \times n_{\text{heads}} d_h$
W_K, W_V	$d \times n_{kv} d_h$
W_1, W_3, $W_2^\top$	$d \times d_{\text{ff}}$
g_1, g_2, g_f	d
W_{head}	$d \times V$

§n today's sections

Ln other lectures.

From 2017 to 2024: what changed in the block

Component	Transformer 2017 / GPT-2 2019	Llama 3 2024	Why
Normalisation	post-norm LayerNorm	**pre-norm RMSNorm**, no bias	stability, gradient flow, cheaper
Positions	sinusoidal / learned absolute	**RoPE**	relative, no parameters, extends
MLP	ReLU or GELU, $d_{\text{ff}} = 4d_{\text{model}}$	**SwiGLU**, $d_{\text{ff}} \approx \frac{8}{3} d_{\text{model}}$	better loss at equal parameters
Attention	multi-head, $n_{kv} = n_q$	**GQA**, $n_{kv} = 8$	KV cache $\div 4$–16
Biases	everywhere	none	no benefit, slower
Dropout	0.1	none	one epoch; nothing to regularise
Embeddings	tied	untied	free at scale
Vocabulary	32k–50k	128k	more languages, fewer tokens
Context	512–1024	8k, then 128k	long-context stage at the end

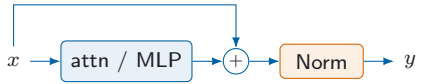
- Every row introduces small loss drop. Together they are worth a few percent of loss, which is a 2–$3\times$ compute saving.
- The *structure* is unchanged: residual stream, attention, MLP, repeated.

Takeaway

The 2024 block is the 2017 block with every unnecessary part removed.

Post-norm (2017) versus pre-norm (GPT-2 onwards)

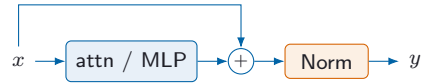
Post-norm: $y = \mathrm{Norm}(x + f(x))$



- The residual sum is normalised in every layer: no identity path from embedding to head.
- Gradients at initialisation are large near the output and vanish towards the input [Xiong et al., 2020]: needs warmup and a small learning rate; deep stacks diverge.

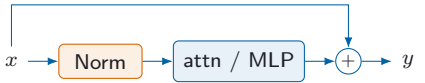
Post-norm (2017) versus pre-norm (GPT-2 onwards)

Post-norm: $y = \text{Norm}(x + f(x))$



- The residual sum is normalised in every layer: no identity path from embedding to head.
- Gradients at initialisation are large near the output and vanish towards the input [Xiong et al., 2020]: needs warmup and a small learning rate; deep stacks diverge.

Pre-norm: $y = x + f(\text{Norm}(x))$



- The residual stream is an *unnormalised sum* $x_0 + \sum_l f_l(\cdot)$: an identity path from embedding to head, gradients reach every layer directly.
- Larger learning rates, little warmup, 100+ layers without tricks. One final norm before the head. Variants: norm on the output (OLMo 2), both (Gemma 2/3).

LayerNorm (2016) versus RMSNorm (2019)

LayerNorm [Ba et al., 2016]

$$\mathrm{LN}(x) = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} \odot g + b$$

- μ, σ^2 over a token's d features: re-centres and re-scales. Two reductions, $2d$ params.
- The 2017 Transformer, BERT, GPT-2/3.

LayerNorm (2016) versus RMSNorm (2019)

LayerNorm [Ba et al., 2016]

$$\mathrm{LN}(x) = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} \odot g + b$$

- μ, σ^2 over a token's d features: re-centres and re-scales. Two reductions, $2d$ params.
- The 2017 Transformer, BERT, GPT-2/3.

RMSNorm [Zhang & Sennrich, 2019]

$$\mathrm{RMS}(x) = \frac{x}{\sqrt{\frac{1}{d}\sum_i x_i^2 + \epsilon}} \odot g$$

- No mean, no bias: re-scales only. One reduction, d parameters, 7–64 % faster.
- T5, Llama, Qwen, DeepSeek, Gemma, Mistral.

LayerNorm (2016) versus RMSNorm (2019)

LayerNorm [Ba et al., 2016]

$$\mathrm{LN}(x) = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} \odot g + b$$

- μ, σ^2 over a token's d features: re-centres and re-scales. Two reductions, $2d$ params.
- The 2017 Transformer, BERT, GPT-2/3.

RMSNorm [Zhang & Sennrich, 2019]

$$\mathrm{RMS}(x) = \frac{x}{\sqrt{\frac{1}{d}\sum_i x_i^2 + \epsilon}} \odot g$$

- No mean, no bias: re-scales only. One reduction, d parameters, 7–64 % faster.
- T5, Llama, Qwen, DeepSeek, Gemma, Mistral.

Takeaway

Re-centring did little for quality, so it was removed: same loss, less work, fewer parameters. Most 2024 changes happened this way.

Positions: add a vector (2017) versus rotate q and k (2021)

Absolute: $x_m \leftarrow x_m + p_m$
once, at the input

- 2017: fixed sinusoids $\sin(m\omega_i), \cos(m\omega_i)$; GPT-2, BERT: a learned p_m per position.
- The score $(x_m + p_m)^\top W_Q^\top W_K (x_n + p_n)$ mixes content with *absolute* position in four cross terms.

Positions: add a vector (2017) versus rotate q and k (2021)

Absolute: $x_m \leftarrow x_m + p_m$
once, at the input

- 2017: fixed sinusoids $\sin(m\omega_i), \cos(m\omega_i)$; GPT-2, BERT: a learned p_m per position.
- The score $(x_m + p_m)^\top W_Q^\top W_K (x_n + p_n)$ mixes content with *absolute* position in four cross terms.

Relative: $q_m^\top k_n = q^\top R((n-m)\theta)\, k$
in every layer

- RoPE [Su et al., 2021]: $q_m = R(m\theta)q$, $k_n = R(n\theta)k$; the score depends on $n - m$ only, v is untouched.
- No parameters; position enters every layer afresh; any length is defined.

Positions: add a vector (2017) versus rotate q and k (2021)

Absolute: $x_m \leftarrow x_m + p_m$
once, at the input

- 2017: fixed sinusoids $\sin(m\omega_i), \cos(m\omega_i)$; GPT-2, BERT: a learned p_m per position.
- The score $(x_m + p_m)^\top W_Q^\top W_K (x_n + p_n)$ mixes content with *absolute* position in four cross terms.

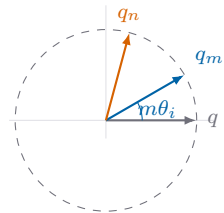
Relative: $q_m^\top k_n = q^\top R((n-m)\theta)\, k$
in every layer

- RoPE [Su et al., 2021]: $q_m = R(m\theta)q$, $k_n = R(n\theta)k$; the score depends on $n - m$ only, v is untouched.
- No parameters; position enters every layer afresh; any length is defined.

	sinusoidal (2017)	learned absolute (GPT-2, BERT)	RoPE (Llama, Qwen, DeepSeek, Gemma)
applied	at the input, once	at the input, once	to q, k in every layer
score depends on	absolute m, n	absolute m, n	$n - m$ only
parameters	0	$L_{\max} \cdot d$	0
beyond $L_{\max}$	untrained	undefined	defined; extendable

Rotary position embeddings (RoPE)

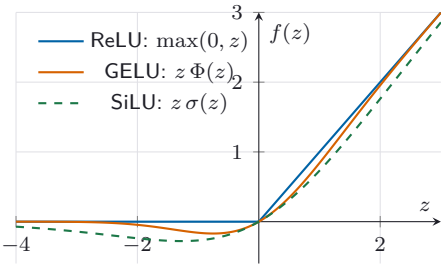
- Attention is permutation-invariant; positions must be injected. Learned absolute vectors (GPT-2) know nothing beyond the trained length.
- RoPE [Su et al., 2021]: rotate each *pair* of q and k dimensions (x_{2i}, x_{2i+1}) at position m by the angle $m\theta_i$, $\theta_i = \text{base}^{-2i/d_{\text{head}}}$. Not applied to v.
- Then $q_m^\top k_n$ depends only on $m - n$: relative positions from a rotation with no parameters.
- Low i: fast rotation, local; high i: wavelengths up to $2\pi \cdot$ base. Base 10 000 in 2021, 500 000 in Llama 3 for 128k context.



Takeaway

Dot product of two rotated vectors depends on the difference of angles.

Activation functions: ReLU, GELU, SiLU



- **ReLU** (2017): a hard gate. Zero output *and zero gradient* for $z < 0$; units can die.
- **GELU** (BERT, GPT-2/3): z times $P(\mathcal{N}(0,1) < z)$. Smooth, a small dip, a gradient everywhere.
- **SiLU**, also swish (Llama, Qwen, DeepSeek): $z\,\sigma(z)$. GELU's shape, cheaper to compute.

The MLP: plain (2017) versus gated, SwiGLU (2020)

Plain: $\mathrm{FFN}(x) = \mathrm{act}(xW_1)\,W_2$



- Two matrices, $d_{\mathrm{ff}} = 4d$: $8d^2$ parameters.
- Each hidden unit gates itself through the activation.

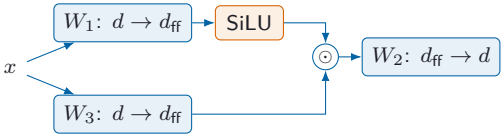
The MLP: plain (2017) versus gated, SwiGLU (2020)

Plain: $\mathrm{FFN}(x) = \mathrm{act}(xW_1)\,W_2$



- Two matrices, $d_{\mathrm{ff}} = 4d$: $8d^2$ parameters.
- Each hidden unit gates itself through the activation.

Gated: $\mathrm{FFN}(x) = \big(\mathrm{SiLU}(xW_1) \odot xW_3\big)\,W_2$



- Three matrices; $d_{\mathrm{ff}} = \frac{8}{3}d$ keeps $8d^2$ parameters [Shazeer, 2020].
- Each hidden unit is gated by a *second* linear view of x, learned.

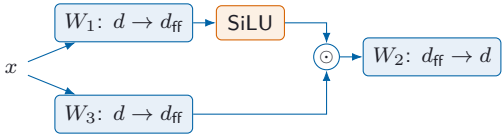
The MLP: plain (2017) versus gated, SwiGLU (2020)

Plain: $\mathrm{FFN}(x) = \mathrm{act}(xW_1)\,W_2$



- Two matrices, $d_{\text{ff}} = 4d$: $8d^2$ parameters.
- Each hidden unit gates itself through the activation.

Gated: $\mathrm{FFN}(x) = \big(\mathrm{SiLU}(xW_1) \odot xW_3\big)\,W_2$



- Three matrices; $d_{\text{ff}} = \frac{8}{3}d$ keeps $8d^2$ parameters [Shazeer, 2020].
- Each hidden unit is gated by a *second* linear view of x, learned.

A few percent lower loss at equal compute across scales, no crisp explanation (the paper: "divine benevolence"). SwiGLU: Llama, Qwen, DeepSeek; GeGLU: Gemma. No biases, no dropout. The MLP holds $8d^2$ of a block's $12d^2$ parameters: two thirds of a dense model, which is what MoE (§5) makes sparse.

Plain versus gated: the controlled experiment

FFN	gate	d_{ff}	log-ppl
ReLU	–	3072	1.997
GELU	–	3072	1.983
Swish (SiLU)	–	3072	1.994
Bilinear	linear	2048	1.960
GLU	sigmoid	2048	1.982
ReGLU	ReLU	2048	1.953
GEGLU	GELU	2048	**1.942**
SwiGLU	SiLU	2048	**1.944**

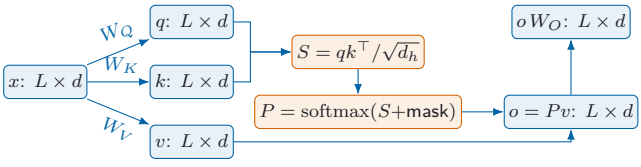
T5-base, C4, $65{,}536$ steps, all else identical; d_{ff} shrunk for gated rows to match parameters and FLOPs. Shazeer (2020), Table 1; noise ± 0.005.

- Only the activation changed ($1.997 \to 1.983$): barely above noise.
- The gate is the effect: every gated row beats every plain row, even the *bilinear* one.
- GEGLU and SwiGLU tie; 0.055 nats $\approx 1.7\times$ the compute here (Kaplan's exponent).
- Replicated at matched compute by Narang et al. (2021): one of few of 26 modifications that transferred.

Takeaway

What "a few percent at equal parameters" means, and why every 2024 model has the gate.

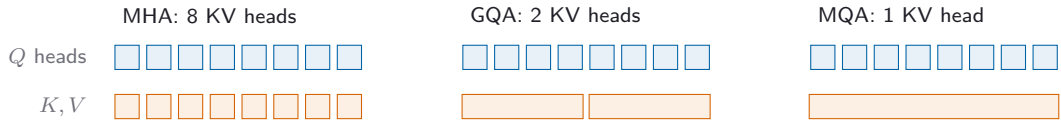
Attention: queries, keys, values



per head: q, k, v are split into n_{heads} groups of $d_h = d/n_{\text{heads}}$ columns; S and P are $L \times L$ for *each* head; the n_{heads} outputs are concatenated before W_O

- Three linear views of the same input: a token's **query** (what it looks for), **key** (what it offers), **value** (what it passes on).
- $S_{ij} = q_i \cdot k_j/\sqrt{d_h}$, one score per pair; the causal mask removes $j > i$; the row-wise softmax makes weights that sum to 1.
- $o_i = \sum_j P_{ij} v_j$: a weighted average of earlier tokens' values. Heads give n_{heads} independent patterns; W_O mixes them.

Attention heads: MHA, GQA, MQA



- n_q query heads share n_{kv} key/value heads; each group of n_q/n_{kv} queries attends to the same K, V [Shazeer, 2019; Ainslie et al., 2023].
- Llama 3: $n_{kv} = 8$ at all sizes, $d_{\text{head}} = 128$. Quality cost of GQA-8: within noise; MQA costs a little.
- **Why**: serving keeps K, V of every past token: $2 \cdot n_{\text{layers}} \cdot n_{kv} \cdot d_{\text{head}} \cdot 2$ bytes. Llama 3 8B: 128 KB per token, 512 KB with MHA; at 128k context 16 vs 64 GB per sequence.

Takeaway

GQA is an inference decision made at pretraining time; most choices since 2023 are.

The block in code: norm, MLP, residuals

```
def lin(x, W):                          # W stored as (out, in), applied to any leading dims
    return # use einsum here as in last lecture
def rmsnorm(x, g, eps=1e-5):
    xf = x.float()                                                  # statistics in fp32
    return # compute as shown in this lecture
def swiglu(x, W1, W2, W3):              # W1, W3: (d_ff, d); W2: (d, d_ff); d_ff = 8/3 d
    return # compute as shown in this lecture
def block(x, p, mask, cos, sin):        # pre-norm; the residual stream is x itself
    ...
    return # compute as shown in this lecture
def lm(ids, P):
    x = P.emb[ids]                                                  # (b, s, d)
    for p in P.blocks: x = block(x, p, mask, cos, sin)
    return ...                           # logits (b, s, V)
```

Everything is a function of tensors; the `nn.Module` wrappers in A1 only hold parameters. Mask and RoPE tables are computed once.

The block in code: RoPE and attention

```
def rope_tables(s, d, base=10000):                          # once per model
    inv = base ** (-torch.arange(0, d, 2) / d)              # (d/2,) frequencies
    ang = torch.arange(s)[:, None] * inv                    # (s, d/2) angles
    return ang.cos()[:, None], ang.sin()[:, None]           # broadcast over heads

def rope(x, cos, sin):                                      # x: (b, s, h, d)
    x1, x2 = x[..., 0::2], x[..., 1::2]
    return torch.stack((x1 * cos - x2 * sin, x1 * sin + x2 * cos), -1).flatten(-2)

def sdpa(q, k, v, mask):                                    # (b, h, s, d)
    ...
    return ...
```

- `attn`: project to q, k, v; split heads; RoPE on q, k; `sdpa`; merge heads; output projection. Causal mask: `torch.tril(torch.ones(s, s, dtype=bool))`.
- This `sdpa` materialises the $s \times s$ scores. Fine for A1; **L4** replaces it.

Hyperparameters and initialisation

1. The modern Transformer block
2. Hyperparameters and initialisation
3. Training stability
4. Attention variants
5. Mixture of experts
6. Reading a model card
7. Wrap-up

Shape hardly matters (within reason)

Knob	GPT-2 small	Llama 3 8B	Llama 3 405B	Sensible range
$d_{\text{ff}}/d_{\text{model}}$	4 (GELU)	3.5 (SwiGLU)	3.25	2.5–4; flat optimum
head dimension	64	128	128	64–128
$d_{\text{model}}/n_{\text{layers}}$ (aspect ratio)	64	128	130	50–200; flat optimum
n_{kv}	12 (MHA)	8	8	4–8
vocabulary	50k	128k	128k	32k–256k, see **L1**

- Kaplan et al. (2020), §3.1: varying depth, width and heads at fixed N changes the loss by a few percent; varying N or D by $10\times$ changes it by tens of percent.
- So: take the shape from the table, then spend your attention on N, D and the learning rate.

Takeaway

Architecture search below the level of "which block" is a rounding error next to scale and data. A2 shows you this in your own numbers.

What does matter: learning rate and batch size

Model	$\alpha_{\max}$	batch (tokens)	schedule
GPT-3 125M	$6 \cdot 10^{-4}$	0.5M	cosine
GPT-3 175B	$6 \cdot 10^{-5}$	3.2M	cosine
Llama 3 8B	$3 \cdot 10^{-4}$	4M	cosine
Llama 3 405B	$8 \cdot 10^{-5}$	4M $\rightarrow$ 16M	cosine
DeepSeek-V3	$2.2 \cdot 10^{-4}$	15M $\rightarrow$ 62M	step decay
A1 model	$\sim 10^{-3}$ (sweep!)	8k–64k	cosine

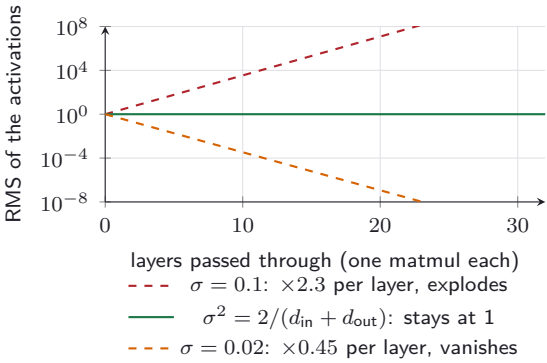
- The optimal rate falls with size, roughly as d_{model}^{-1}; μP makes this exact (**L6**). The optimal batch grows during training.

- Loss vs. rate is a U with a broad bottom on a log axis: sweep $\{1, 2, 5\} \times 10^k$.
- Too high: spikes, divergence. Too low: the same loss takes 2–3$\times$ the tokens.
- Weight decay 0.1, $\beta_2 = 0.95$, clipping at 1.0: fixed, not swept.

A1

Sweep $\alpha_{\max}$ over at least four values; plot final validation loss against it.

Initialisation: keep the signal at unit scale



A stack of 512×512 matmuls, $\mathcal{N}(0, \sigma^2)$ weights, no norms or residuals: each layer scales the signal by $\sigma\sqrt{d_{\text{in}}}$ (gradients likewise). $d_{\text{in}}, d_{\text{out}}$: a weight matrix's input and output widths.

- **A1**: $\mathcal{N}(0, \sigma^2)$, $\sigma^2 = 2/(d_{\text{in}} + d_{\text{out}})$, truncated at $\pm 3\sigma$; embeddings $\mathcal{N}(0, 1)$, gains 1.
- **GPT-2**: $\sigma = 0.02$, output projections $\times 1/\sqrt{2n_{\text{layers}}}$ so the stream does not grow with depth.
- Pre-norm renormalises every sub-layer input: an init off by $2\times$ still trains, just slower.

Takeaway

Check at step 0: activations of order 1, loss $\ln V$. Init still matters in μP (**L6**).

Reading a config: Llama 3 8B, parameter count by hand

$d_{\text{model}} = 4096$, $n_{\text{layers}} = 32$, $n_q = 32$, $n_{kv} = 8$, $d_{\text{head}} = 128$, $d_{\text{ff}} = 14336$, $V = 128256$, untied.

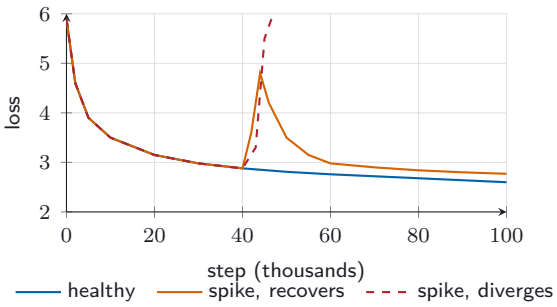
	per block	total
W_q, W_o: $2 \cdot 4096^2$	33.6M	
W_k, W_v: $2 \cdot 4096 \cdot 1024$	8.4M	
MLP: $3 \cdot 4096 \cdot 14336$	176.2M	
32 blocks		6.98B
embedding + head: $2 \cdot 128256 \cdot 4096$		1.05B
		8.03B

- 81 % of the block parameters are the MLP; GQA made attention 19 %.
- N for $6ND$: 7.0B, not 8.0B.
- Two minutes per model card; catches most misreadings (active vs total, tied vs untied, d_{ff} conventions).

Training stability

1. The modern Transformer block
2. Hyperparameters and initialisation
3. Training stability
4. Attention variants
5. Mixture of experts
6. Reading a model card
7. Wrap-up

What instability looks like



- Spikes appear after thousands of good steps; the gradient norm spikes a few steps *earlier*. Log it.
- More frequent with size and learning rate. Manual recovery (skip the batch, restart from a checkpoint) was normal in 2022; the goal now is not to need it.
- Two mechanisms explain most spikes: attention logits growing without bound, output logits drifting [Wortsman et al., 2023].

The stability toolkit

Tool	What it does	Fixes	Cost
Pre-norm	identity residual path	most of the 2017 problems	none
Warmup	small steps while Adam's v is noisy	early divergence	a few hundred steps
Clipping at 1.0	caps one bad batch	isolated spikes	none
QK-norm	RMSNorm on q and k before the dot product	attention-logit growth	tiny (Gemma 3, OLMo 2)
z-loss, $10^{-4} \log^2 Z$	keeps the softmax normaliser near 1	output-logit drift	one scalar per token (PaLM)
fp32 softmax and norms	no bf16 rounding where it hurts	silent degradation	some memory traffic
$\beta_2 = 0.95$	shorter memory in v	spikes after rare batches	none

Takeaway

The 2025 default: all of the above (OLMo 2, Gemma 3); costs nothing measurable, removes most spikes. A1 uses the first three; add QK-norm if your sweep spikes.

Attention variants

1. The modern Transformer block
2. Hyperparameters and initialisation
3. Training stability
4. Attention variants
5. Mixture of experts
6. Reading a model card
7. Wrap-up

Why change attention at all

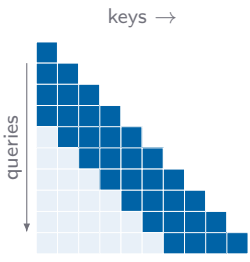
- Compute: score FLOPs $4Ld_{\text{model}}$ per token per block, $L/6d_{\text{model}}$ of the rest. Irrelevant at 4k, dominant at 128k.
- Memory during training: the $L \times L$ matrix per head; solved exactly by FlashAttention (**L4**).
- Memory at inference: the KV cache, read *in full* for every generated token. This is the one that hurts.
- Quality: softmax attention is the best mixer per FLOP at short range and the only one with exact recall at long range.

The menu

- Keep softmax, shrink the cache: GQA, MLA.
- Keep softmax, restrict the range: sliding windows, sparse attention.
- Replace softmax: linear attention, SSMs; then hybridise.

All in production in 2025. None has replaced full attention entirely.

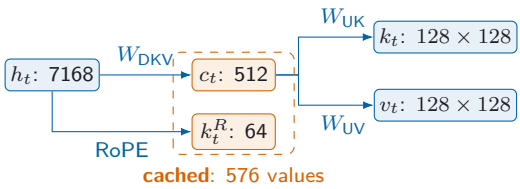
Sliding window and local/global layers



window 4 (dark) inside the causal mask (light)

- Each query attends to the last w keys: cost and cache $O(Lw)$ instead of $O(L^2)$; information still travels w positions per layer [Jiang et al., 2023].
- Mistral 7B: $w = 4096$ everywhere. Gemma 2/3: alternate **local** ($w = 1024$) and **global** layers, 5:1 in Gemma 3, so one global layer in six carries the long-range cache [Gemma Team, 2025].
- Sparse attention (DeepSeek NSA and DSA, 2025) generalises this: each query selects a few blocks of keys with a small learned index and skips the rest.

Multi-head latent attention (DeepSeek-V2/V3)



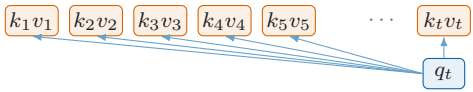
- Compress the token to a 512-dim latent c_t; all heads' K, V are up-projections of it, recomputed on the fly. Cache c_t, not K, V [DeepSeek-AI, 2024].
- RoPE does not commute with the up-projection: a 64-dim rotary key is cached too. At decode time $W_{\text{UK}}, W_{\text{UV}}$ fold into W_Q, W_O.

Takeaway

128 heads at $57\times$ less cache than MHA; matches MHA in quality.

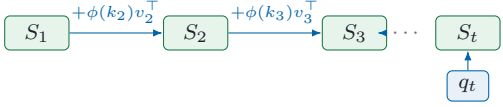
Linear attention and state-space models

softmax attention: the cache is every past (k, v)



memory $O(L)$, work $O(L)$ per token; exact recall of any past token

linear attention / SSM: one fixed-size state $S_t \in \mathbb{R}^{d \times d}$



memory $O(d^2)$, work $O(d^2)$ per token; the state cannot hold L arbitrary facts

- Without the softmax the sum is a **recurrence** [Katharopoulos et al., 2020]; Mamba [Gu & Dao, 2023] adds token-dependent gates and a parallel scan.
- Same box, same limit: exact recall (copying, lookups) degrades with length.

Hybrids

Mostly linear blocks, a few attention blocks: Jamba (1 in 8), Qwen3-Next (3:1). Linear cost, attention recall.

Mixture of experts

1. The modern Transformer block
2. Hyperparameters and initialisation
3. Training stability
4. Attention variants
5. Mixture of experts
6. Reading a model card
7. Wrap-up

The idea: decouple parameters from compute

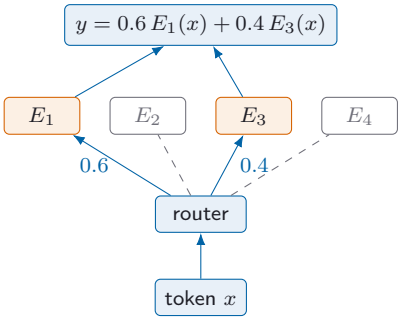
- Dense: every token passes through every parameter; FLOPs per token $= 2N$. Parameters and compute are locked together.
- MoE: replace each MLP by E MLPs (**experts**) and a **router** that sends each token to k of them. Parameters $\propto E$; FLOPs $\propto k$ [Shazeer et al., 2017].
- DeepSeek-V3: 671B parameters, 37B active per token; 14.8T tokens for $2.8 \cdot 10^6$ H800-hours. The FLOPs of a 37B dense model, the capacity of something larger.
- At equal training FLOPs an MoE typically reaches the loss of a dense model 2–$4\times$ its active size.

Key idea

Most parameters of a dense model are MLP weights a given token never needed. MoE only computes the ones it needs.

- Attention stays dense and shared; only the MLPs are split.
- **Active** parameters go into $6ND$ and inference FLOPs; **total** parameters are the memory bill.

Routing: how a token picks its experts



- Router: $s = W_r x$, one score per expert; keep the top-k; gates g_i from a softmax (or normalised sigmoid) over the kept scores.
- $y = \sum_{i \in \text{top-}k} g_i \, E_i(x)$; the E_i are ordinary SwiGLU MLPs applied to x.
- Discrete choice: unchosen experts get no gradient; the router learns only through the gates.
- Mixtral: $E = 8, k = 2$. DeepSeek-V3: $E = 256, k = 8$ plus one **shared expert** every token uses.

Load balancing, capacity, and communication

- Left alone, routers collapse: a few experts win early, get more gradient, win more.
- **Auxiliary loss** [Fedus et al., 2022]: $\alpha E \sum_i f_i P_i$, f_i the fraction of tokens sent to expert i, P_i its mean router probability; minimal when uniform. Too strong hurts quality.
- DeepSeek-V3: no auxiliary loss; a per-expert **bias** added to the scores before top-k, nudged up for underused experts after each step.
- **Capacity**: with expert parallelism each expert has a fixed buffer; tokens beyond it are dropped (skip the MLP) or the buffer is dynamic ("dropless").

The systems view

Experts are spread over GPUs. Every MoE layer is two **all-to-all** exchanges: tokens to their experts and back. Routing decides the communication pattern (**L5**).

- Balance matters twice: for learning, and for not waiting on the busiest GPU.

The design space, in numbers

Model	total	active	E	k	shared	note
Mixtral 8x7B (2023)	47B	13B	8	2	–	8 large experts, Mistral 7B attention
DeepSeek-V3 (2024)	671B	37B	256	8	1	fine-grained experts, MLA, first 3 layers dense
Llama 4 Maverick (2025)	400B	17B	128	1	1	alternating dense and MoE layers
Qwen3-235B-A22B (2025)	235B	22B	128	8	–	
gpt-oss-120b (2025)	117B	5.1B	128	4	–	36 layers, open weights

- The trend: **many small experts**, high sparsity ($k/E \approx 3\,\%$), often a shared expert; small experts specialise better [Dai et al., 2024].
- Total/active ratios of 10–$25\times$ are normal in 2025.

Takeaway

Read an MoE card twice: active parameters for compute and speed, total parameters for memory and price.

What MoE costs, and when it wins

Costs

- **Memory**: all experts resident. DeepSeek-V3 in bf16 is 1.34 TB: 17 H100s just to hold the weights, before any cache.
- **Batch size**: each expert sees k/E of the tokens. To be compute-bound (**L2**) it needs ~ 300 of its own, so the global batch must be E/k times larger. Small-batch serving of an MoE is memory-bound on all 671B of weights.
- **Communication**: two all-to-alls per layer, custom kernels. **Training**: routing instabilities, fiddlier fine-tuning.

MoE wins when training compute is the constraint and memory is not, and for high-throughput serving with large batches: the frontier labs.

Dense wins on one GPU, at small batch, on devices, and for simplicity: Llama 3 8B, Gemma 3, A1, A2, most research.

Takeaway

MoE trades memory and engineering for FLOPs. The frontier took the trade; the open dense 8B still exists for a reason.

Reading a model card

1. The modern Transformer block
2. Hyperparameters and initialisation
3. Training stability
4. Attention variants
5. Mixture of experts
6. Reading a model card
7. Wrap-up

Five model cards side by side

	GPT-2 small	Llama 3 8B	Llama 3 70B	Llama 3 405B	DeepSeek-V3
year	2019	2024	2024	2024	2024
parameters (active)	124M	8.0B	70.6B	405B	671B (37B)
n_{layers}	12	32	80	126	61
d_{model}	768	4096	8192	16384	7168
heads n_q/n_{kv}	12 / 12	32 / 8	64 / 8	128 / 8	128 (MLA)
d_{ff}	3072	14336	28672	53248	2048×256
vocabulary	50257	128256	128256	128256	129280
context (train)	1024	8k $\rightarrow$ 128k	8k $\rightarrow$ 128k	8k $\rightarrow$ 128k	4k $\rightarrow$ 128k
block	LN, GELU	RoPE, SwiGLU	RoPE, SwiGLU	RoPE, SwiGLU	RoPE, SwiGLU
training tokens	$\sim$10B	15T	15T	15T	14.8T

- Down a column: $d_{\text{model}}/n_{\text{layers}} \approx 100$–$130$, $d_{\text{head}} = 128$, $n_{kv} = 8$: the same shape rules at every scale.
- Across the row: what changed since GPT-2 is the block details, the vocabulary, the context, and $1000\times$ the tokens.

What architecture aspects are changing

Unchanged since 2017

- The residual stream as the model's working memory.
- Attention mixes across positions, an MLP within a position.
- Next-token prediction with a softmax over a fixed vocabulary.
- Dense matmuls as the unit of compute.

Moving in 2025–26

- Hybrids and sparse attention chip at the L^2 term.
- MoE everywhere at the frontier
- fp8 training, 4-bit inference (**L7**); multimodal inputs through the same block (**L10**).

Wrap-up

1. The modern Transformer block
2. Hyperparameters and initialisation
3. Training stability
4. Attention variants
5. Mixture of experts
6. Reading a model card
7. Wrap-up

Summary

1. The 2024 block: pre-norm RMSNorm, RoPE, SwiGLU, GQA, no biases, no dropout. Each change is small; together they are worth 2–$3\times$ compute. A1 implements this block.
2. Shape hardly matters within known ranges; N, D, the learning rate and the batch size do. Sweep the learning rate, fix the rest.
3. Stability is a checklist: warmup, clipping, QK-norm, z-loss, fp32 where it counts.
4. Attention variants are inference decisions: GQA and MLA shrink the cache, windows and sparsity shrink the range, linear and SSM blocks remove the cache and lose exact recall; hybrids combine them.
5. MoE decouples parameters from FLOPs: active parameters set compute, total parameters set memory; routing balance is the engineering problem.

References

- Vaswani et al. (2017). Attention is all you need. *NeurIPS*.
- Radford et al. (2019). Language models are unsupervised multitask learners (GPT-2). OpenAI.
- Grattafiori et al. (2024). The Llama 3 herd of models. arXiv:2407.21783.
- Xiong et al. (2020). On layer normalization in the Transformer architecture (pre-LN). *ICML*.
- Zhang & Sennrich (2019). Root mean square layer normalization. *NeurIPS*.
- Su et al. (2021). RoFormer: Enhanced Transformer with rotary position embedding. arXiv:2104.09864.
- Shazeer (2020). GLU variants improve Transformer. arXiv:2002.05202.
- Shazeer (2019). Fast Transformer decoding: One write-head is all you need (MQA). arXiv:1911.02150.
- Ainslie et al. (2023). GQA: Training generalized multi-query Transformer models from multi-head checkpoints. *EMNLP*.
- Kaplan et al. (2020). Scaling laws for neural language models. arXiv:2001.08361.
- Wortsman et al. (2023). Small-scale proxies for large-scale Transformer training instabilities. arXiv:2309.14322.
- OLMo Team (2024). 2 OLMo 2 Furious. arXiv:2501.00656.
- Gemma Team (2025). Gemma 3 technical report. arXiv:2503.19786.
- Jiang et al. (2023). Mistral 7B. arXiv:2310.06825.

References (cont.)

- DeepSeek-AI (2024). DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model (MLA). arXiv:2405.04434.
- DeepSeek-AI (2024). DeepSeek-V3 technical report. arXiv:2412.19437.
- Katharopoulos et al. (2020). Transformers are RNNs: Fast autoregressive Transformers with linear attention. *ICML*.
- Gu & Dao (2023). Mamba: Linear-time sequence modeling with selective state spaces. arXiv:2312.00752.
- Lieber et al. (2024). Jamba: A hybrid Transformer–Mamba language model. arXiv:2403.19887.
- Shazeer et al. (2017). Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *ICLR*.
- Fedus, Zoph & Shazeer (2022). Switch Transformers: Scaling to trillion parameter models with simple and efficient sparsity. *JMLR*.
- Jiang et al. (2024). Mixtral of experts. arXiv:2401.04088.
- Dai et al. (2024). DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models. arXiv:2401.06066.
- Dao et al. (2022). FlashAttention: Fast and memory-efficient exact attention with IO-awareness. *NeurIPS*.